

PROJECT SYNOPSIS

NAT GATE WAY FOR PRIVATE SUBNET INTERNET ACCESS

Enabling Secure Outbound-Only for EC2 Instances in a Private Subnet

Project Domain : Cloud Computing/ Amazon Web Services (AWS)

Project Type : Academic Project

Submitted by :

Sr. no.	Team Member	Enrollment ID
1	Tushit Mishra	2415001691
2	Prashant Sharma	2415001154
3	Dev Jaisawat	2415000495
4	Skand Singh	2415001577

Technology Stack :

Amazon VPC • Public & Private Subnets • Internet Gateway • NAT Gateway • Elastic IP
• Route Tables • Amazon EC2 • Security Groups • AWS Management Console

Project Synopsis

1. Abstract

Cloud workloads are rarely uniform in their exposure requirements: some resources, such as web servers or bastion hosts, must be reachable from the internet, while others, such as application servers, databases or internal processing nodes, should be able to reach the internet for updates and API calls but must never be directly reachable from it. Amazon VPC allows this distinction to be enforced structurally through subnet design and routing rather than through security-group rules alone. This project proposes and demonstrates a complete AWS-based workflow for provisioning a NAT (Network Address Translation) Gateway that gives an EC2 instance in a private subnet outbound-only internet access, while a matching public-subnet instance retains full inbound and outbound reachability.

In the proposed workflow, a custom VPC is built from scratch with one public and one private subnet in separate availability zones. An Internet Gateway is attached to the VPC and associated with the public subnet's route table, giving public-subnet resources direct two-way internet access. A NAT Gateway is then provisioned inside the public subnet using an allocated Elastic IP and the private subnet's route table is updated to send all outbound traffic (0.0.0.0/0) through that NAT Gateway rather than through the Internet Gateway. Two EC2 instances are launched - PublicEC2, acting as a bastion host with a public IP and PrivateEC2, launched with no public IP at all and security groups are chained so that PrivateEC2 only accepts SSH from PublicEC2, never from the open internet.

The project concludes with deliberate negative and positive testing: PrivateEC2 is proven able to reach the internet (via ping and package installation through yum) purely through the NAT Gateway, while a direct SSH attempt from an external machine to PrivateEC2's IP address is proven to time out, confirming that no inbound path exists. The result is a demonstrable, reference-pattern VPC architecture that gives practical exposure to subnetting, routing, NAT translation and layered network security.

Keywords

- AWS
- Amazon VPC
- NAT Gateway
- Internet Gateway
- Public and Private Subnets
- Route Tables
- Elastic IP
- Network Segmentation

2. Project Summary

The project is focused on designing and building a two-tier VPC network - one public subnet and one private subnet and configuring a NAT Gateway so that an EC2 instance with no public IP address can still reach the internet for outbound purposes such as software updates. The final environment is intended to demonstrate a working, verifiable network boundary: outbound traffic from the private

instance succeeds, while inbound traffic from the internet directly to the private instance is structurally blocked at the routing layer, not merely at the security-group layer.

3. Introduction

Amazon Virtual Private Cloud (VPC) allows an organization to define an isolated, logically separated section of the AWS cloud in which resources can be launched with fine-grained control over networking. A common and recommended design pattern within a VPC is to separate resources into public and private subnets according to whether they need to be directly reachable from the internet. Public subnets host resources such as load balancers or bastion hosts that require direct inbound access, while private subnets host application or database resources that should remain shielded from direct inbound exposure.

A challenge arises when a private-subnet resource still needs outbound internet access - for example to download OS updates or contact an external API - without being reachable from the internet in return. AWS solves this with a NAT Gateway: a managed, highly available address-translation service placed in a public subnet that lets private-subnet instances initiate outbound connections while blocking any connection initiated from the internet side. This project builds and verifies that exact pattern end-to-end.

3.1 Need for the Project

- To understand how public and private subnets are separated within a VPC using route tables.
- To learn the role of an Internet Gateway versus a NAT Gateway.
- To configure a NAT Gateway correctly, including its dependency on an Elastic IP and a public subnet.
- To practically verify that outbound-only access behaves as intended, rather than assuming a configuration is correct.
- To understand security-group chaining as a second layer of access control alongside routing.
- To gain hands-on experience with AWS VPC networking fundamentals.

3.2 Scope

The scope covers a single VPC within one AWS account and region: creating public and private subnets, attaching an Internet Gateway, building and associating two route tables, allocating an Elastic IP, provisioning a NAT Gateway, launching two EC2 instances (one public, one private), configuring chained security groups and validating both outbound connectivity and inbound isolation. The project is designed as an academic demonstration and can later be extended with a NAT instance cost comparison, multi-AZ NAT Gateway redundancy, VPC endpoints for AWS service traffic and Infrastructure-as-Code automation.

4. Problem Statement

An EC2 instance placed in a private subnet, by design, has no public IP address and no route to an Internet Gateway, which means it cannot install software updates, download packages or reach external APIs on its own. Simply attaching an Internet Gateway to the private subnet would solve

connectivity but would also make the instance directly reachable from the internet, defeating the purpose of placing it in a private subnet in the first place.

The project addresses this problem by introducing a NAT Gateway as an intermediary: the private subnet's route table sends outbound traffic to the NAT Gateway instead of the Internet Gateway and the NAT Gateway - which lives in the public subnet and holds its own Elastic IP - performs the address translation and forwards the request onward through the Internet Gateway. Because the NAT Gateway only relays connections that were initiated from inside the VPC, no unsolicited inbound connection can reach the private instance and this is validated directly rather than assumed.

5. Objective

- Understand the fundamentals of VPC subnetting and route table association.
- Create a custom VPC (myVPC) with a defined CIDR block.
- Create one public subnet and one private subnet in separate availability zones.
- Create and attach an Internet Gateway to the VPC.
- Build a Public Route Table sending 0.0.0.0/0 to the Internet Gateway, associated with the public subnet.
- Build a Private Route Table with only the local route initially, associated with the private subnet.
- Enable auto-assign public IPv4 on the public subnet.
- Allocate an Elastic IP address.
- Create a NAT Gateway in the public subnet using the allocated Elastic IP.
- Update the Private Route Table to send 0.0.0.0/0 to the NAT Gateway.
- Launch private EC2 instance with no public IP and a public EC2 instance acting as a bastion host.
- Chain security groups so the private instance only accepts SSH from the public instance.
- Verify outbound internet access from the private instance via ping and package installation.
- Verify that the private instance remains unreachable via a direct inbound connection attempt.

5.1 Subnet and Routing Configuration

Parameter	Public Subnet	Private Subnet
Name	Public Subnet	PrivateSubnet
CIDR Block	10.0.1.0/24	10.0.2.0/24
Availability Zone	ap-south-1a	ap-south-1b
Auto-assign Public IP	Enabled	Disabled
Route Table	PublicRouteTable	PrivateRouteTable
Default Route (0.0.0.0/0)	Internet Gateway (myIGW)	NAT Gateway (MyNATGateway)

6. Technologies and AWS Services Used

Parameter	Roles in Project
-----------	------------------

Amazon VPC	Provides the isolated network (myVPC, 10.0.0.0/16) in which all resources are launched.
Public & Private Subnets	Segment the VPC into an internet-facing zone and an internet-isolated zone.
Internet Gateway (IGW)	Gives the public subnet direct two-way internet connectivity.
NAT Gateway	Translates and forwards outbound-only traffic from the private subnet to the internet.
Elastic IP	Provides the fixed public IP address the NAT Gateway uses to reach the internet.
Route Tables	Enforce, at the network layer, which subnet can reach the internet and how.
Amazon EC2	Hosts PublicEC2 (bastion) and PrivateEC2 (isolated workload).
Security Groups	Add a second, instance-level layer of access control via SG chaining.
AWS Management Console	Used to perform and document every configuration step.

6.1 Why a NAT Gateway Rather than a Public IP on the Private Instance

Assigning a public IP directly to the private instance, or routing its subnet through the Internet Gateway, would technically provide internet access but would also expose the instance to unsolicited inbound connections from anywhere on the internet. A NAT Gateway solves this asymmetrically: it allows connections that originate from inside the private subnet to reach the internet and receive replies, but it does not allow any connection to be initiated from the internet side, because it has no mechanism to route an unsolicited inbound packet back to a specific private instance.

6.2 Why two separate Route Tables ?

Using one route table per subnet, rather than a single shared table, makes the internet-access posture of each subnet explicit and independently auditable. The Public Route Table's 0.0.0.0/0 → Internet Gateway rule and the Private Route Table's 0.0.0.0/0 → NAT Gateway rule cannot be confused with one another, and the private subnet is provably isolated from the internet at every point before the NAT Gateway route is added, since only the default local route exists until Phase 03 of the workflow.

7. Proposed Solution / Architecture

The proposed architecture places a single VPC (myVPC, 10.0.0.0/16) across two subnets in two availability zones. The public subnet (10.0.1.0/24, ap-south-1a) hosts the NAT Gateway and the bastion EC2 instance, and is routed directly to the internet through an Internet Gateway. The private subnet (10.0.2.0/24, ap-south-1b) hosts the isolated EC2 instance and is routed to the internet only through the NAT Gateway sitting in the public subnet.

7.1 Logical Traffic Flow

Outbound traffic from PrivateEC2 flows: Private Subnet → PrivateRouteTable → NAT Gateway → Public Subnet → Internet Gateway → Internet. No inbound path exists back to PrivateEC2 - the NAT Gateway only relays responses to connections that PrivateEC2 itself initiated.

7.2 Implementation Phases

Phase	Focus	Steps
-------	-------	-------

01-Network Foundation	VPC + public and private subnets	Steps 1-3
02-Routing & Gateway	Internet Gateway + route tables	Steps 4-6
03-NAT Gateway	Auto-assign IP + Elastic IP + NAT Gateway + route update	Steps 7-10
04-Compute Layer	Launch private and public EC2 instances	Steps 11-14
05-Verification	Test outbound access, confirm inbound is blocked	Steps 14-17

7.3 Core Design Principle

The architecture separates two responsibilities that are easy to conflate: reachability (whether a resource can be reached from outside) and connectivity (whether a resource can reach outside). Subnet placement and route tables govern reachability, while the NAT Gateway specifically grants connectivity without granting reachability. Security groups then add a further, independent layer - PrivateSG only accepts SSH from PublicSG, so even an operator with console access to the private instance must first pass through the bastion host.

8. Project Workflow and Implementation

Step 1 – Create the VPC: A custom VPC named myVPC was created with CIDR block 10.0.0.0/16, no IPv6 CIDR block, and default tenancy.

Step 2 – Create the Public Subnet: PublicSubnet was created with CIDR 10.0.1.0/24 in availability zone ap-south-1a.

Step 3 – Create the Private Subnet: PrivateSubnet was created with CIDR 10.0.2.0/24 in availability zone ap-south-1b.

Step 4 – Create the Internet Gateway: myIGW was created and attached to myVPC, with its state verified as Attached.

Step 5 – Create the Public Route Table: PublicRouteTable was created under myVPC, given a route of 0.0.0.0/0 → myIGW, and associated with PublicSubnet.

Step 6 – Create the Private Route Table: PrivateRouteTable was created under myVPC and associated with PrivateSubnet, deliberately left with only the default local route (10.0.0.0/16 → local) at this stage, so the private subnet remained fully isolated from the internet.

Step 7 – Enable Auto-Assign Public IP: Auto-assign public IPv4 addressing was enabled on PublicSubnet so instances launched there automatically receive a public IP.

Step 8 – Allocate an Elastic IP: A new Elastic IP address was allocated from Amazon's IPv4 pool with default settings.

Step 9 – Create the NAT Gateway: MyNATGateway was created inside PublicSubnet with Zonal availability mode, Public connectivity type, and the previously allocated Elastic IP, then verified to reach the Available state.

Step 10 – Update the Private Route Table: A new route (0.0.0.0/0 → MyNATGateway) was added to PrivateRouteTable, giving the private subnet outbound-only internet access while keeping the local route intact.

Step 11 – Launch PrivateEC2: An Amazon Linux 2023, t3.micro instance named PrivateEC2 was launched into PrivateSubnet with Auto-assign Public IP disabled and PrivateSG attached, then verified Running with no public IPv4 address and a private IPv4 in the 10.0.2.x range.

Step 12 – Launch PublicEC2: An Amazon Linux 2023, t3.micro instance named PublicEC2 was launched into PublicSubnet with Auto-assign Public IP enabled and PublicSG attached, then verified Running with both a public IPv4 and a private IPv4 in the 10.0.1.x range.

Step 13 – Connect to PublicEC2: PublicEC2 was accessed via EC2 Instance Connect using its public IPv4 address, and root access was confirmed.

Step 14 – Create the Key File on PublicEC2: The private key (MyKeyPair.pem) was recreated on PublicEC2 using nano, saved, and its permissions restricted with chmod 400, so PublicEC2 could act as an SSH jump host into PrivateEC2.

Step 15 – SSH into PrivateEC2 via PublicEC2: From PublicEC2, an SSH session was opened to PrivateEC2's private IP address using the recreated key file, confirming that the bastion path works as intended.

Step 16 – Test Internet via the NAT Gateway: From inside PrivateEC2, three outbound tests were run — a successful ping to google.com, a successful sudo yum update -y and a successful sudo yum install httpd -y — confirming that outbound internet access through the NAT Gateway works correctly for both ICMP and package-manager traffic.

Step 17 – Verify Inbound Is Still Blocked: From a local machine outside AWS, a direct SSH attempt to PrivateEC2's IP address timed out, confirming that PrivateEC2 has no reachable inbound path from the internet despite its working outbound access.

8.1 Expected Sequence

VPC Created → Subnets Created → Internet Gateway Attached → Route Tables Configured → Elastic IP Allocated → NAT Gateway Available → Private Route Updated → Instances Launched → Bastion Path Verified → Outbound Access Confirmed → Inbound Access Confirmed Blocked.

9. Detailed Component Roles

9.1 Amazon VPC

The VPC (myVPC, 10.0.0.0/16) provides the isolated address space in which every other resource in the project is created, and is the boundary within which subnetting and routing decisions are enforced.

9.2 Public and Private Subnets

PublicSubnet and PrivateSubnet divide the VPC's address space by availability zone and by internet-access posture; only the public subnet is designed to hold internet-facing infrastructure such as the NAT Gateway and the bastion host.

9.3 Internet Gateway

myIGW is the sole component that connects the VPC to the public internet; only subnets whose route tables point to it can be reached from, or reach, the internet directly.

9.4 NAT Gateway

MyNATGateway sits in the public subnet, uses an Elastic IP as its outward-facing address and forwards outbound requests from the private subnet, translating source addresses so replies can find their way back - without ever accepting a connection initiated from outside.

9.5 Route Tables

PublicRouteTable and PrivateRouteTable are the actual enforcement mechanism for the entire architecture: changing a single route (0.0.0.0/0's target) is what converts a subnet's internet-access posture between direct, NAT-mediated, or fully isolated.

9.6 Security Groups

PublicSG and PrivateSG add an instance-level layer beneath the network layer: PrivateSG is configured to accept SSH only from PublicSG, so even lateral access within the VPC is constrained to a single, auditable hop.

10. Networking and Security

All resources operate inside myVPC, with PrivateEC2 retaining only a private IP address throughout the project and never being assigned a public IPv4 address at any stage. PublicEC2 is deliberately exposed as a bastion host so that administrative access to PrivateEC2 is possible without weakening PrivateEC2's own network isolation. The security-group chain - SSH allowed to PublicSG from Anywhere, and SSH allowed to PrivateSG only from PublicSG - means that even if an operator obtains PrivateEC2's private IP address, a direct connection attempt from outside the VPC still fails at the network layer before security groups are even evaluated.

- Use IAM policies to restrict which users can modify VPC, subnet, route table and NAT Gateway configuration.
- Do not rely on the NAT Gateway's outbound-only behavior as a substitute for least-privilege IAM access control.
- Log configuration changes using AWS CloudTrail so changes to route tables and security groups are auditable.
- Keep PrivateSG's inbound rules limited strictly to PublicSG as the source, never to 0.0.0.0/0.
- For production use, consider NAT Gateways in multiple availability zones to avoid a single point of failure.

11. Testing and Validation

Testing was built into the final phase of the workflow itself: each network-layer claim (outbound works, inbound doesn't) was validated with a real connection attempt rather than being inferred from configuration alone.

ID	Test Action	Expected Result
TC-01	Create VPC and verify state	VPC shows state = Available
TC-02	Create public and private subnets	Both subnets show state = Available

TC-03	Attach Internet Gateway to VPC	IGW state = Attached
TC-04	Associate PublicRouteTable with PublicSubnet	0.0.0.0/0 → IGW route active
TC-05	Associate PrivateRouteTable with PrivateSubnet	Only local route present initially
TC-06	Allocate Elastic IP	New public IPv4 address appears
TC-07	Create NAT Gateway	NAT Gateway state = Available
TC-08	Add NAT route to PrivateRouteTable	0.0.0.0/0 → NAT Gateway route active
TC-09	Launch PrivateEC2	Running, no public IP, private IP in 10.0.2.x
TC-10	Launch PublicEC2	Running, public IP present, private IP in 10.0.1.x
TC-11	SSH from PublicEC2 to PrivateEC2	Session succeeds via private IP
TC-12	Ping google.com from PrivateEC2	Replies received via NAT Gateway
TC-13	sudo yum update -y on PrivateEC2	Completes successfully
TC-14	sudo yum install httpd -y on PrivateEC2	Package installed successfully
TC-15	SSH from external machine directly to PrivateEC2	Connection times out

11.1 Monitoring Checklist

- VPC, subnet, and NAT Gateway state at each workflow stage
- Route table entries for both PublicRouteTable and PrivateRouteTable
- Auto-assign public IP setting on each subnet
- Public and private IP address fields on both EC2 instances
- Result of outbound tests (ping, yum) run from PrivateEC2
- Result of the external inbound SSH attempt to PrivateEC2

11.2 Success Criteria

The project is considered successful when PrivateEC2 has no public IP yet can successfully ping external hosts and install packages via the NAT Gateway, and a direct SSH attempt to PrivateEC2 from outside the VPC times out, confirming no inbound path exists.

12. Challenges and Troubleshooting

Problem	Troubleshooting Approach
NAT Gateway creation fails or has no route out	Confirm the NAT Gateway was created in the public subnet and that an Elastic IP was successfully attached before creation.
PrivateEC2 has no internet access after setup	Verify the 0.0.0.0/0 → NAT Gateway route was actually added and saved on PrivateRouteTable, not just previewed.
Cannot SSH from PublicEC2 into PrivateEC2	Re-check that PrivateSG's inbound SSH rule references PublicSG as the source, and that the .pem file's permissions were set with chmod 400.
PrivateEC2 unexpectedly reachable from the internet	Confirm PrivateSubnet's route table was never associated with the Internet Gateway, and that PrivateEC2 truly has no public IPv4 address.

Elastic IP allocation succeeds but NAT Gateway shows pending	Allow a short provisioning delay; NAT Gateways typically take a few minutes to reach the Available state.
--	---

13. Advantages

- Gives private-subnet resources outbound internet access without any direct inbound exposure.
- Enforces isolation at the network/routing layer, which is harder to misconfigure than security-group rules alone.
- Uses a fully managed, highly available AWS service, requiring no patching or maintenance of NAT software.
- Demonstrates a standard, AWS-recommended reference architecture applicable to real production VPC designs.
- Security-group chaining adds a second, independent layer of defense beyond routing.
- Bastion-host pattern (PublicEC2) allows controlled administrative access without weakening PrivateEC2's isolation.

14. Limitations

- A single NAT Gateway in one availability zone is a potential single point of failure for outbound traffic.
- NAT Gateways incur an hourly charge plus per-GB data processing charges, unlike a self-managed NAT instance.
- The project is scoped to a single VPC with one public and one private subnet, not a multi-tier or multi-AZ production layout.
- Protection is limited to network-layer isolation and does not address OS-level vulnerabilities or credential compromise.
- AWS resources used in this project may incur charges depending on usage duration and region.

15. Expected Results and Outcomes

Condition	Expected Observation
VPC and subnets created	Both subnets show state = Available under myVPC
Internet Gateway attached	IGW state = Attached, associated with myVPC
Public route table configured	0.0.0.0/0 → IGW active, associated with PublicSubnet
Private route table configured (pre-NAT)	Only local route present; subnet fully isolated
NAT Gateway created	State = Available, Elastic IP attached
Private route table updated (post-NAT)	0.0.0.0/0 → NAT Gateway active, alongside local route
PrivateEC2 launched	Running, no public IP, reachable only via PublicEC2
Outbound tests from PrivateEC2	ping and yum commands succeed via the NAT Gateway
External inbound SSH attempt to PrivateEC2	Connection times out — no direct path exists

16. Future Scope

- Deploy redundant NAT Gateways across multiple availability zones for high availability.
- Automate the entire VPC build using AWS CLI or Infrastructure-as-Code (CloudFormation/Terraform).
- Add VPC endpoints for S3 and other AWS services to reduce NAT Gateway data-processing costs.
- Integrate CloudWatch alarms and VPC Flow Logs to monitor NAT Gateway usage and detect anomalies.
- Apply IAM policies restricting who can modify route tables and security groups.
- Extend the design to Auto Scaling Groups of private-subnet instances behind the same NAT Gateway.

17. Project Deliverables

- A custom VPC with one public and one private subnet across two availability zones.
- An attached Internet Gateway and two correctly associated route tables.
- A working NAT Gateway backed by an allocated Elastic IP.
- A private EC2 instance with verified outbound-only internet access.
- A public EC2 instance configured as a bastion host with chained security groups.
- Console screenshots documenting all seventeen workflow steps.
- Final project synopsis, assignment report and presentation.
- Version-controlled repository containing all project artifacts.

18. Conclusion

This project demonstrates a NAT Gateway architecture in Amazon VPC, providing a private EC2 instance with outbound-only internet access while preventing direct inbound connections. By configuring subnets, route tables, an Internet Gateway, and NAT Gateway, and validating connectivity through ping, package installation, and an external SSH timeout, the project demonstrates subnetting, routing, address translation, and security. The architecture provides a foundation for multi-AZ redundancy, IaC automation and centralized monitoring.

19. References / Learning Resources

- Amazon Web Services (AWS) documentation for Amazon VPC.
- Amazon Web Services documentation for NAT Gateways.
- Amazon Web Services documentation for Internet Gateways and route tables.
- Amazon Web Services documentation for Elastic IP addresses.
- Amazon Web Services documentation for security groups and EC2 instance connectivity.
- Amazon Web Services pricing documentation for NAT Gateway data processing.

End of Project Synopsis